

1. What is HTTP?

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between a client and a server.

It defines how a client sends a request and how a server returns a response.

Basic flow

Client

|

| HTTP Request

▼

Backend Server

|

| HTTP Response

▼

Client

Example

React Frontend

↓

GET /api/users

↓

Node.js + Express

↓

Database

↓

JSON Response

↓

React Frontend

2. HTTP in Backend Development

HTTP is the foundation of:

- REST APIs
- Web APIs
- Authentication
- File uploads
- Webhooks
- Microservices communication
- Browser-server communication
- API gateways
- Reverse proxies

A Node.js backend commonly uses HTTP through:

```
import http from "node:http";

const server = http.createServer((req, res) => {
  res.writeHead(200, {
    "Content-Type": "application/json"
  });

  res.end(JSON.stringify({
    message: "Hello World"
  }));
});

server.listen(5000);
```

Express is built on top of Node.js HTTP functionality.

3. HTTP Request

An HTTP request is sent from the client to the server.

Example:

```
GET /api/users?page=1 HTTP/1.1
```

```
Host: example.com
```

```
Accept: application/json
```

```
Authorization: Bearer TOKEN
```

An HTTP request contains:

1. Method
2. URL/path
3. HTTP version
4. Headers
5. Body — optional

4. HTTP Request Structure

METHOD + PATH + HTTP VERSION

HEADERS

BODY

Example:

```
POST /api/users HTTP/1.1
```

```
Host: api.example.com
```

```
Content-Type: application/json
```

```
Authorization: Bearer abc123
```

```
{  
  "name": "Uttam",  
  "email": "uttam@example.com"  
}
```

5. HTTP Methods

HTTP methods describe the operation the client wants to perform.

Method	Purpose	Example
GET	Retrieve data	Get users
POST	Create resource / submit data	Create user
PUT	Replace resource	Replace profile
PATCH	Partially modify resource	Update email
DELETE	Delete resource	Delete user
HEAD	Get headers without response body	Check resource
OPTIONS	Ask what communication is supported	CORS preflight
CONNECT	Establish tunnel	Proxy communication
TRACE	Diagnostic loop-back	Debugging

6. GET

Used to retrieve data.

```
GET /api/users
```

Express:

```
app.get("/api/users", (req, res) => {  
  res.json({  
    users: []  
  });  
});
```

GET should generally not be used to modify server state.

7. POST

Used to create a resource or submit data.

POST /api/users

Content-Type: application/json

```
{  
  "name": "Uttam"  
}
```

Express:

```
app.post("/api/users", (req, res) => {  
  console.log(req.body);  
  
  res.status(201).json({  
    message: "User created"  
  });  
});
```

8. PUT

PUT generally replaces the complete representation of a resource.

PUT /api/users/10

```
{  
  "name": "Uttam",  
  "email": "uttam@example.com",  
  "role": "developer"
```

```
}
```

9. PATCH

PATCH performs a partial modification.

```
PATCH /api/users/10
```

```
{  
  "role": "backend developer"  
}
```

PUT vs PATCH

PUT

Complete replacement
Usually sends complete
resource
/users/10

PATCH

Partial modification
Sends only changed
fields
/users/10

10. DELETE

Used to remove a resource.

```
DELETE /api/users/10
```

Express:

```
app.delete("/api/users/:id", async (req, res) => {  
  // delete user  
  
  res.status(204).send();  
});
```

11. HTTP Safe Methods

A method is considered **safe** when it is intended primarily for retrieving information rather than changing server state.

Common safe methods:

GET

HEAD

OPTIONS

For example:

GET /api/products

should not delete or modify products.

12. Idempotent HTTP Methods

An operation is idempotent when making the same request multiple times has the same intended effect as making it once.

Commonly idempotent:

GET

HEAD

PUT

DELETE

OPTIONS

TRACE

POST is generally **not idempotent**.

Example:

DELETE /users/10

Calling it repeatedly should leave user 10 deleted.

But:

POST /orders

could create multiple orders if submitted multiple times.

13. HTTP Headers

Headers provide metadata about the request or response.

Example:

Content-Type: application/json

Authorization: Bearer token

Accept: application/json

Important headers:

Header	Purpose
Host	Target host
Content-Type	Format of body
Content-Length	Body size
Accept	Expected response format
Authorization	Authentication credentials
User-Agent	Client information
Origin	Request origin
Referer	Referring URL
Cookie	Sends cookies
Set-Cookie	Creates/updates cookies
Cache-Control	Caching rules
ETag	Resource version identifier
Location	Redirect/resource location
Allow	Supported methods

14. Content-Type

Content-Type tells the server what format the request body uses.

Example:

Content-Type: application/json

Common types:

application/json

application/x-www-form-urlencoded

multipart/form-data

text/plain

text/html

application/xml

JSON API:

Content-Type: application/json

15. Accept

Accept tells the server which response formats the client can process.

Accept: application/json

The server may respond:

Content-Type: application/json

16. HTTP Request Body

The request body contains data sent to the server.

Example:

```
{  
  "username": "uttam",  
  "password": "secret"  
}
```

Express:

```
app.use(express.json());
```

```
app.post("/login", (req, res) => {  
  console.log(req.body);  
});
```

17. HTTP Response

The server sends an HTTP response to the client.

Example:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{  
  "message": "Success"  
}
```

A response contains:

6. Status code
7. Headers
8. Body — optional

18. HTTP Status Codes

HTTP status codes contain three digits.

1xx → Informational

2xx → Success

3xx → Redirection

4xx → Client Error

5xx → Server Error

19. 1xx Status Codes

Informational responses.

Examples:

100 Continue

101 Switching Protocols

103 Early Hints

Most application developers rarely handle these directly.

20. 2xx Success

200 OK

Request succeeded.

GET /users

→ 200 OK

201 Created

Resource successfully created.

POST /users

→ 201 Created

202 Accepted

Request accepted for processing but processing may not be complete.

Useful for asynchronous jobs.

204 No Content

Successful request with no response body.

Example:

```
DELETE /users/10
```

→ 204 No Content

21. 3xx Redirection

Common:

301 Moved Permanently

302 Found

303 See Other

304 Not Modified

307 Temporary Redirect

308 Permanent Redirect

301

Permanent redirect.

302

Temporary/general redirect.

304

Resource has not changed, so cached representation can be reused.

307

Temporary redirect while preserving the request method.

308

Permanent redirect while preserving the request method.

22. 4xx Client Errors

Common:

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

405 Method Not Allowed

408 Request Timeout

409 Conflict

410 Gone

413 Content Too Large

415 Unsupported Media Type

422 Unprocessable Content

429 Too Many Requests

23. 400 Bad Request

Request is invalid.

Example:

```
{  
  "email": "invalid-email"  
}
```

Backend:

```
return res.status(400).json({  
  message: "Invalid request"  
});
```

24. 401 Unauthorized

Means authentication is required or authentication credentials are invalid.

Example:

GET /api/profile

Authorization: Bearer invalid-token

Response:

401 Unauthorized

25. 403 Forbidden

The server understood the request but refuses to authorize it.

Example:

Normal User → Admin API

↓

403

401 vs 403

401 → Authentication problem

403 → Authorization/permission problem

26. 404 Not Found

Resource or endpoint was not found.

```
GET /api/users/999999
```

Response:

404 Not Found

27. 409 Conflict

Used when the request conflicts with the current server state.

Example:

User registration

Email already exists

↓

409 Conflict

28. 422 Unprocessable Content

The request syntax may be valid, but the submitted data fails validation/business rules.

Example:

```
{  
  "age": -10  
}
```

29. 429 Too Many Requests

Used for rate limiting.

Client

↓

1000 requests

↓

Rate limiter

↓

429 Too Many Requests

30. 5xx Server Errors

Common:

500 Internal Server Error

501 Not Implemented

502 Bad Gateway

503 Service Unavailable

504 Gateway Timeout

500

Unexpected error in your backend.

502

Proxy/gateway received an invalid response from an upstream server.

503

Service temporarily unavailable.

504

Gateway/proxy did not receive a timely response from upstream.

31. URL Structure

Example:

`https://api.example.com:443/api/users/10?active=true`

Parts:

`https`

↓

Protocol

`api.example.com`

↓

Host

`443`

↓

Port

`/api/users/10`

↓

Path

`?active=true`

↓

Query string

32. Path Parameters

Used to identify a specific resource.

GET `/api/users/10`

Express:

```
app.get("/api/users/:id", (req, res) => {  
  console.log(req.params.id);  
});
```

33. Query Parameters

Used for filtering, sorting, pagination, searching, etc.

GET /api/users?page=2&limit=10&role=admin

Express:

```
app.get("/api/users", (req, res) => {  
  console.log(req.query.page);  
  console.log(req.query.limit);  
  console.log(req.query.role);  
});
```

34. Path Parameter vs Query Parameter

Path Parameter	Query Parameter
Identifies resource	Filters/configures result
Usually required for resource	Often optional
/users/10	/users?page=2
req.params	req.query

35. HTTP Cookies

Cookies are small pieces of data associated with a domain.

Example:

Set-Cookie: sessionId=abc123; HttpOnly; Secure

Browser sends:

Cookie: sessionId=abc123

Cookies are commonly used for:

- Sessions
- Authentication
- Preferences
- Tracking

36. Important Cookie Attributes

HttpOnly

Prevents JavaScript from directly accessing the cookie.

HttpOnly

Useful against some XSS-based cookie theft.

Secure

Cookie is sent only over HTTPS.

Secure

SameSite

Controls cross-site cookie sending.

Values:

Strict

Lax

None

For SameSite=None, browsers require Secure.

Max-Age

Specifies lifetime in seconds.

Domain

Specifies applicable domain.

Path

Specifies URL path scope.

37. Session-Based Authentication

Typical flow:

Login

↓

Server validates credentials

↓

Server creates session

↓

Session ID stored in cookie

↓

Browser sends cookie on future requests

↓

Server identifies user

Example:

Cookie: sessionId=abc123

38. Token-Based Authentication

Example:

Authorization: Bearer eyJhbGci...

Backend:

```
const authHeader = req.headers.authorization;
```

```
const token = authHeader?.split(" ")[1];
```

Typical flow:

Login

↓

Server creates access token

↓

Client sends token

↓

Backend verifies token

↓

Request authorized

39. Authorization Header

Common format:

Authorization: Bearer TOKEN

Used with:

- JWT

- OAuth 2.0 access tokens
- API tokens

40. HTTP Authentication vs Authorization

Authentication

"Who are you?"

Login

JWT

Session

OAuth

Authorization

"What are you allowed to do?"

Admin → Delete users

User → View own profile

41. CORS

CORS = Cross-Origin Resource Sharing

It controls whether browser JavaScript from one origin can access resources from another origin.

Example:

Frontend:

`http://localhost:5173`

Backend:

`http://localhost:5000`

Different origins → CORS may apply.

42. Origin

An origin consists of:

scheme + host + port

Example:

`http://localhost:5173`

Changing any of these can create a different origin.

43. Simple CORS Request

Example:

```
fetch("https://api.example.com/users");
```

For requests meeting the browser's CORS-safelisted conditions, the browser can send the actual request without a preflight.

44. Preflight Request

For certain cross-origin requests, the browser first sends:

```
OPTIONS /api/users HTTP/1.1
```

```
Origin: https://frontend.example.com
```

```
Access-Control-Request-Method: POST
```

```
Access-Control-Request-Headers: content-type, authorization
```

Server can respond:

```
Access-Control-Allow-Origin: https://frontend.example.com
```

```
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
```

Access-Control-Allow-Headers: Content-Type, Authorization

Then the browser may send the actual request.

45. CORS in Express

```
import cors from "cors";
```

```
app.use(cors({  
  origin: "https://frontend.example.com",  
  methods: ["GET", "POST", "PUT", "DELETE"],  
  allowedHeaders: ["Content-Type", "Authorization"]  
}));
```

Never blindly allow every production origin unless that is intentionally appropriate.

46. HTTP Caching

Caching stores previously obtained data so it can be reused.

Benefits:

- Faster responses
- Reduced bandwidth
- Reduced server load
- Better scalability

Important headers:

Cache-Control

ETag

Last-Modified

If-None-Match

If-Modified-Since

Expires

47. Cache-Control

Example:

Cache-Control: max-age=3600

Means a cached response can be considered fresh for the specified period.

Common directives:

public

private

no-cache

no-store

max-age

must-revalidate

Important distinction

no-cache

does not mean "do not store".

It generally means the cache must validate freshness before reuse.

no-store

means don't store the response.

48. ETag

ETag identifies a particular representation/version of a resource.

Server:

ETag: "abc123"

Client later sends:

If-None-Match: "abc123"

If unchanged:

304 Not Modified

This saves transferring the unchanged response body.

49. HTTP Compression

HTTP responses can be compressed.

Common algorithms:

gzip

br (Brotli)

Client:

Accept-Encoding: gzip, br

Server:

Content-Encoding: br

Compression reduces response size and can improve network performance.

50. HTTP Content Negotiation

Client tells server what it can accept.

Example:

Accept: application/json

Accept-Language: en-US

Accept-Encoding: gzip, br

Server selects an appropriate representation.

51. HTTP Keep-Alive

HTTP connections can be reused for multiple requests.

Instead of:

Request → Connection

Response → Close

Request → New Connection

Response → Close

connection reuse allows:

Connection

|— Request

|— Response

|— Request

|— Response

└— Request

This reduces connection overhead.

52. HTTP/1.1

HTTP/1.1 is widely supported and introduced important improvements over HTTP/1.0.

Features include:

- Persistent connections
- Host header
- Chunked transfer encoding
- Better caching
- Pipelining support

Traditional HTTP/1.1 has request/response ordering limitations that can contribute to head-of-line blocking at the application connection level.

53. HTTP/2

HTTP/2 improves transport efficiency while retaining HTTP semantics.

Important features:

- Binary framing
- Multiplexing
- Header compression using HPACK
- Stream prioritization mechanisms
- Multiple concurrent streams over a connection

Example:

One TCP Connection

|— Request A
|— Request B
|— Request C
|— Request D
└— Request E

54. HTTP/3

HTTP/3 uses **QUIC** instead of TCP.

Basic stack:

HTTP/1.1 → TCP

HTTP/2 → TCP

HTTP/3 → QUIC → UDP

HTTP/3 provides features such as:

- Multiplexed streams
- Reduced connection establishment latency
- Better behavior when packets are lost on one stream
- Connection migration support

55. HTTP Version Comparison

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC/UDP
Binary framing	No	Yes	Yes
Multiplexing	Limited	Yes	Yes
Header compression	No	HPACK	QPACK
Modern web performance	Good	Better	Better in many network conditions

56. HTTP vs HTTPS

HTTP

Client

↓

HTTP

↓

Server

HTTP itself does not provide encryption or authentication of the server.

HTTPS

Client

↓

TLS

↓

HTTP

↓

Server

HTTPS is HTTP carried over a secure TLS connection.

57. What is HTTPS?

HTTPS = HTTP Secure

It protects communication using **TLS (Transport Layer Security)**.

HTTPS provides:

1. Encryption
2. Server authentication
3. Integrity protection

58. Why HTTPS is Important

Without encryption, an attacker on an unsafe network could potentially observe or manipulate traffic.

HTTPS protects sensitive communication such as:

- Passwords
- Authentication tokens
- Personal information
- Payment-related data
- API requests
- API responses

59. TLS

TLS is the cryptographic protocol used to secure HTTPS connections.

Modern systems should use current secure TLS versions, primarily:

TLS 1.3

TLS 1.2

Old protocols such as SSLv2/SSLv3 should not be used.

60. TLS Handshake — Basic Concept

Before normal encrypted HTTP communication:

Client

|

| ClientHello

▼

Server

|

| ServerHello + Certificate

▼

Client

|

| Key agreement

▼

Secure encrypted connection

|

▼

HTTP Request / Response

TLS 1.3 uses modern key exchange and reduces handshake overhead compared with older TLS designs.

61. Digital Certificate

A TLS certificate helps the client verify the identity of the server.

Example:

`api.example.com`

↓

TLS Certificate

↓

Certificate Authority

Certificate contains information such as:

- Domain names
- Public key
- Issuer
- Validity period
- Digital signature

62. Certificate Authority

A **Certificate Authority (CA)** is a trusted organization that signs certificates.

Examples include:

- Let's Encrypt
- DigiCert
- Sectigo
- GlobalSign

The browser/operating system maintains trusted CA information.

63. Public Key and Private Key

TLS certificates contain/publicize a public key.

The server keeps its private key secret.

Public Key



Can be distributed

Private Key



Must remain secret

The exact cryptographic operations vary by TLS version and cipher suite, but the private key must never be exposed.

64. Encryption Types

Symmetric encryption

Same secret key is used for encryption/decryption.

Key



Encrypt



Data



Decrypt



Same key

Examples:

AES

ChaCha20

Asymmetric cryptography

Uses a public/private key pair.

Examples:

RSA

ECDSA

EdDSA

Modern TLS uses asymmetric cryptography primarily for authentication/key establishment and symmetric cryptography for bulk data encryption.

65. HTTPS Does Not Mean "Everything Is Secure"

HTTPS protects data **in transit**.

It does not automatically protect against:

- SQL injection
- XSS
- Broken authorization
- Insecure password storage
- Server vulnerabilities
- Malicious application logic
- Compromised server

You still need secure backend development.

66. HTTP Redirect to HTTPS

Production websites often redirect:

`http://example.com`

↓

301/308

↓

`https://example.com`

Example Express concept:

```
app.use((req, res, next) => {  
  if (req.secure) {  
    return next();  
  }  
  
  return res.redirect(  
    `https://${req.headers.host}${req.originalUrl}`  
  );  
});
```

When behind a trusted reverse proxy, configure Express proxy handling correctly before relying on `req.secure`.

67. HSTS

HSTS = HTTP Strict Transport Security

Header:

`Strict-Transport-Security: max-age=31536000; includeSubDomains`

It tells browsers to use HTTPS for the domain for the specified period.

Important:

HSTS should be deployed carefully, especially with `includeSubDomains` and preload-related settings.

68. Secure Cookies

For HTTPS authentication cookies:

Set-Cookie: sessionId=abc123; Secure; HttpOnly; SameSite=Lax

Important:

Secure

HttpOnly

SameSite

These attributes improve cookie security.

69. TLS Termination

In production, TLS may terminate at:

Browser

↓ HTTPS

Load Balancer / Reverse Proxy

↓ HTTP or HTTPS

Node.js

Examples:

Nginx

Cloudflare

AWS Load Balancer

Kubernetes Ingress

The backend must correctly understand proxy headers and trust boundaries.

70. Reverse Proxy

A reverse proxy sits in front of your backend.

Internet

↓

Nginx

↓

Node.js

↓

Database

Responsibilities can include:

- TLS termination
- Load balancing
- Compression
- Routing
- Rate limiting
- Static file serving
- Security headers

71. HTTP and Node.js

Node.js provides the built-in HTTP module.

```
import http from "node:http";
```

```
const server = http.createServer((req, res) => {  
  console.log(req.method);  
  console.log(req.url);  
  
  res.writeHead(200, {  
    "Content-Type": "application/json"
```

```
});

res.end(JSON.stringify({
  message: "Hello"
}));
});

server.listen(5000);
```

72. Express and HTTP

Express simplifies HTTP server development.

```
app.get("/users", (req, res) => {
  res.status(200).json({
    users: []
  });
});
```

Express provides convenient handling for:

- Routing
- Middleware
- Request parsing
- Responses
- Error handling

73. Express Request Object

Important properties:

```
req.method
req.url
```

```
req.params  
req.query  
req.body  
req.headers  
req.cookies  
req.ip
```

Example:

```
app.get("/users/:id", (req, res) => {  
  console.log(req.method);  
  console.log(req.params.id);  
  console.log(req.query);  
  console.log(req.headers);  
  
  res.json({  
    message: "Success"  
  });  
});
```

74. Express Response Object

Common methods:

```
res.status()  
res.json()  
res.send()  
res.end()  
res.redirect()  
res.cookie()
```

```
res.clearCookie()  
res.set()
```

Example:

```
res  
  .status(200)  
  .json({  
    success: true  
  });
```

75. HTTP Middleware

Middleware executes during request processing.

Request

↓

Logger

↓

CORS

↓

Authentication

↓

Validation

↓

Controller

↓

Response

Example:

```
app.use((req, res, next) => {  
  console.log(req.method, req.url);  
  next();  
});
```

76. Request Lifecycle

Typical Express API:

Client

↓

DNS

↓

TCP/TLS

↓

Reverse Proxy

↓

Node.js

↓

Express

↓

Middleware

↓

Router

↓

Controller

↓

Service

↓

Database



Response



Client

Understanding this lifecycle is extremely important for backend development.

77. REST API

REST is an architectural style commonly implemented using HTTP.

Example:

GET /api/users

POST /api/users

GET /api/users/:id

PATCH /api/users/:id

DELETE /api/users/:id

78. REST API Best Practices

Prefer nouns for resources:

/api/users

/api/products

/api/orders

Rather than:

/api/getUsers

/api/createUser

/api/deleteUser

Use HTTP methods to express the operation.

79. API Versioning

Example:

```
/api/v1/users
```

```
/api/v2/users
```

Versioning helps when introducing breaking API changes.

80. Pagination

Instead of returning thousands of records:

```
GET /api/users?page=1&limit=20
```

Response:

```
{
  "data": [],
  "page": 1,
  "limit": 20,
  "total": 200
}
```

For very large datasets, cursor-based pagination can be more efficient.

81. Rate Limiting

Rate limiting restricts how many requests a client can make.

Example:

100 requests / minute

If exceeded:

429 Too Many Requests

Useful for:

- Login endpoints
- OTP endpoints
- Public APIs
- Expensive operations
- Abuse prevention

82. Request Timeout

Servers should avoid allowing requests to remain open indefinitely.

Possible timeout sources:

Client timeout

Reverse proxy timeout

Load balancer timeout

Node.js/server timeout

Database query timeout

Example client:

```
const controller = new AbortController();

setTimeout(() => controller.abort(), 5000);

fetch(url, {
  signal: controller.signal
```

```
});
```

Production systems should configure sensible timeouts across layers.

83. Webhooks

A webhook allows one server to notify another server through HTTP.

Example:

Payment Service

↓

POST /api/webhooks/payment

↓

Your Backend

Important considerations:

- Verify signatures
- Authenticate webhook source
- Validate payload
- Make processing idempotent
- Return appropriate status codes
- Handle retries

84. HTTP File Upload

File uploads commonly use:

Content-Type: multipart/form-data

Example:

Client

↓

multipart/form-data

↓

Express

↓

File storage

Possible storage:

Local filesystem

AWS S3

Cloudinary

Object storage

Never blindly trust uploaded file names or MIME types.

85. HTTP Streaming

Instead of sending the entire response at once, data can be streamed.

Useful for:

- Large files
- Video/audio
- Server-generated reports
- Long-running responses

Node.js supports streams.

86. Chunked Transfer Encoding

HTTP/1.1 can send a response in chunks when the total content length is not known beforehand.

Conceptually:

Response

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

↓

End

HTTP/2 and HTTP/3 use their own framing mechanisms rather than HTTP/1.1 chunked transfer encoding.

87. HTTP Long Polling

Client sends a request and server keeps it open until data becomes available or a timeout occurs.

Client — Request —→ Server

Client ← data/timeout — Server

Then the client sends another request.

88. Server-Sent Events

SSE allows a server to send a stream of events to a browser over an HTTP connection.

Client — connection —→ Server

Client ← event — Server

Client ← event — Server

Client ← event — Server

Useful for:

- Notifications
- Live feeds
- Progress updates

89. WebSocket vs HTTP

HTTP	WebSocket
Request/response	Persistent bidirectional connection
Client normally initiates request	Either side can send messages
REST APIs	Chat/live communication
Request-based	Event/message-based
WebSocket connections begin with an HTTP-based handshake but then upgrade to the WebSocket protocol.	

90. HTTP Connection vs WebSocket

HTTP:

Client → Request

Client ← Response

Client → Request

Client ← Response

WebSocket:

Client ↔ Connection ↔ Server

91. DNS and HTTP

Before connecting to a domain:

`https://api.example.com`

↓

DNS lookup

↓

IP address



TCP/QUIC connection



TLS for HTTPS



HTTP request

Understanding this flow helps diagnose production networking problems.

92. HTTP Request Journey

A simplified HTTPS request:

Browser



DNS



IP address



TCP connection



TLS handshake



HTTP request



Reverse proxy



Node.js



Express



Business logic



Database



HTTP response



Browser

For HTTP/3, QUIC replaces the TCP + TLS transport arrangement.

93. HTTP Security Headers

Useful production headers include:

Strict-Transport-Security

Content-Security-Policy

X-Content-Type-Options

Referrer-Policy

Permissions-Policy

Example:

X-Content-Type-Options: nosniff

Security headers should be configured according to the application's requirements.

94. CSRF

CSRF = Cross-Site Request Forgery

An attacker attempts to cause a victim's browser to make an unwanted authenticated request.

Risk is particularly relevant when authentication relies on cookies automatically sent by the browser.

Defenses can include:

- SameSite cookies
- CSRF tokens
- Origin/Referer validation where appropriate
- Proper application architecture

95. XSS

XSS = Cross-Site Scripting

Attacker-controlled JavaScript is injected into a page and executed in another user's browser.

Backend defenses include:

- Validate input
- Encode output appropriately
- Use Content Security Policy
- Avoid unsafe HTML rendering
- Set appropriate security headers

96. SQL Injection

Attacker manipulates database queries through untrusted input.

Bad:

```
const query = `SELECT * FROM users WHERE id = ${req.query.id}`;
```

Use parameterized queries instead.

Concept:

```
db.query(  
  "SELECT * FROM users WHERE id = $1",  
  [req.query.id]
```

);

97. HTTPS + Authentication

A secure production flow:

HTTPS

↓

Login

↓

Password verification

↓

Session/JWT

↓

Secure authentication transport

↓

Authorization

HTTPS protects credentials/tokens while they travel over the network, but you must still store passwords securely and implement authentication correctly.

98. HTTP Error Handling

Do not expose internal errors.

Bad:

```
{  
  "error": "MongoServerError: ..."  
}
```

Better:

```
{  
  "success": false,  
  "message": "Internal server error"  
}
```

Log detailed errors on the server.

99. API Error Format

A consistent format is useful.

Example:

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": [  
    {  
      "field": "email",  
      "message": "Invalid email"  
    }  
  ]  
}
```

100. HTTP Logging

Production backends should log useful request information.

Example:

POST /api/users 201 45ms

GET /api/products 200 18ms

GET /api/orders/10 404 12ms

Tools:

Morgan

Pino

Winston

OpenTelemetry-compatible observability tools

Be careful not to log:

- Passwords
- Access tokens
- Session secrets
- Sensitive personal data

101. HTTP Observability

Monitor:

Request count

Error rate

Latency

Throughput

Status codes

Timeouts

Database latency

Useful metrics:

p50 latency

p95 latency

p99 latency

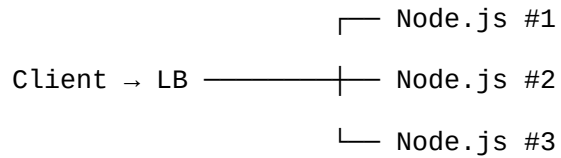
5xx rate

4xx rate

Requests per second

102. HTTP and Load Balancing

Multiple backend instances can be placed behind a load balancer.



Benefits:

- Scalability
- High availability
- Traffic distribution

103. Stateless HTTP Backend

HTTP itself is stateless.

A request should contain enough information for the server to process it, or the server uses an external/shared state mechanism.

For scalable APIs:

Client

↓

Load Balancer

├─ Server 1

├─ Server 2

└─ Server 3

Shared state may be stored in:

PostgreSQL

MongoDB

Redis

104. HTTP + Redis

Redis can support:

- Sessions
- Caching
- Rate limiting
- Distributed locks
- Temporary data

Example:

Client

↓

Node.js

↓

Redis

↓

Database

105. HTTP and Database Transactions

HTTP request:

POST /api/orders

Backend:

Validate request

↓

Authenticate user



Begin transaction



Create order



Update inventory



Commit



HTTP 201

If something fails:

Rollback



HTTP error

HTTP itself does not provide database transaction semantics.

106. HTTP Request Validation

Never trust client input.

Validate:

Body

Query parameters

Path parameters

Headers

Uploaded files

Example:

```
if (!req.body.email) {  
  return res.status(400).json({  
    message: "Email is required"  
  });  
}
```

Production applications commonly use validation libraries such as:

Zod

Joi

Yup

Ajv

107. HTTP Security Checklist

For a production backend:

- ✓ Use HTTPS
- ✓ Use secure TLS configuration
- ✓ Redirect HTTP to HTTPS where appropriate
- ✓ Enable HSTS after careful deployment
- ✓ Validate all input
- ✓ Use authentication
- ✓ Implement authorization
- ✓ Rate-limit sensitive endpoints
- ✓ Use secure cookies when applicable
- ✓ Protect against CSRF when applicable
- ✓ Prevent SQL/NoSQL injection

- ✓ Configure CORS carefully
- ✓ Limit request body size
- ✓ Set sensible timeouts
- ✓ Use security headers
- ✓ Don't expose stack traces
- ✓ Don't log secrets
- ✓ Keep dependencies updated
- ✓ Monitor errors and latency

108. HTTP Request Size Limits

Attackers can send very large payloads.

Express example:

```
app.use(express.json({  
  limit: "1mb"  
}));
```

Choose limits according to your application's actual requirements.

109. HTTP Method Override

Some systems use method override when clients cannot directly send methods such as PATCH or DELETE.

Example concept:

```
POST /users/10  
_method=DELETE
```

Use only when actually required by the application architecture.

110. HTTP OPTIONS

OPTIONS asks what communication options are supported by a resource/server.

Example:

```
OPTIONS /api/users
```

It is also commonly used by browsers for CORS preflight requests.

111. HEAD

HEAD behaves similarly to GET but does not return the response body.

Useful for checking:

- Resource existence
- Metadata
- Content length
- Cache information

Example:

```
HEAD /video.mp4
```

112. HTTP TRACE

TRACE can be used for diagnostic purposes.

It is generally disabled in production environments because it can create security concerns and is rarely required by applications.

113. HTTP CONNECT

CONNECT is primarily used to establish a tunnel through a proxy, commonly for HTTPS traffic.

Conceptually:

```
Client
```

```
↓ CONNECT
```

Proxy

↓

Target server

114. HTTP Proxy

A forward proxy acts on behalf of clients.

Client

↓

Forward Proxy

↓

Internet

A reverse proxy acts on behalf of servers.

Internet

↓

Reverse Proxy

↓

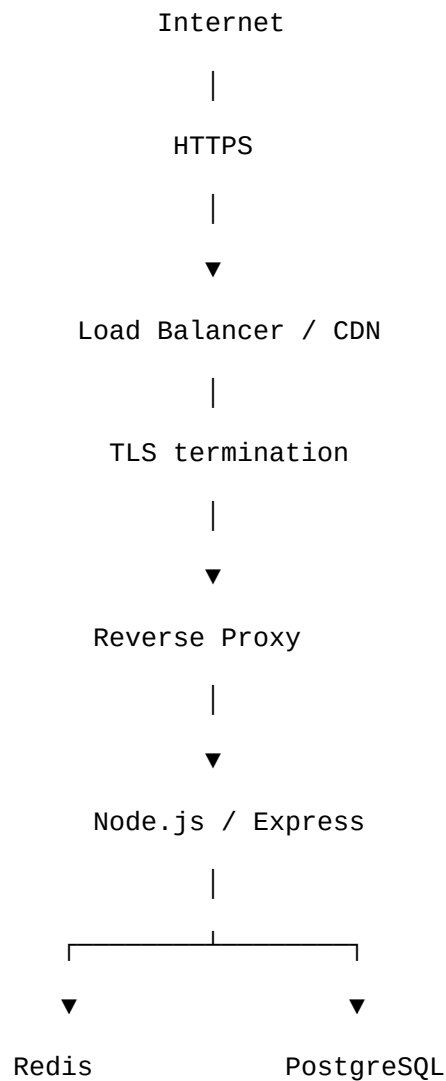
Backend

115. Forward Proxy vs Reverse Proxy

Forward Proxy	Reverse Proxy
Represents clients	Represents servers
Client-side	Server-side
Internet access/filtering	Routing/load balancing
Corporate proxy	Nginx/load balancer

116. Production HTTPS Architecture

A common architecture:



117. HTTP vs HTTPS — Interview Table

HTTP	HTTPS
HTTP protocol	HTTP over TLS
No transport encryption by itself	Encrypted transport
Default port 80	Default port 443
Vulnerable to network interception	Protects data in transit
Suitable for local/internal use when	Preferred for production internet traffic

appropriate

No server authentication from HTTP itself

TLS certificates authenticate server
identity

118. HTTP vs HTTPS — Simple Example

HTTP:

Client

↓

GET /login

↓

Network

↓

Server

HTTPS:

Client

↓

TLS encrypted connection

↓

GET /login

↓

Server

119. HTTP vs HTTPS Interview Question

Q: Is HTTPS a different protocol from HTTP?

Answer:

HTTPS is essentially HTTP communicated over a TLS-secured connection. HTTP defines the application-level request/response semantics, while TLS provides encryption, integrity protection, and server authentication for the connection.

120. Simple Request vs Preflighted Request

Simple

Browser



Actual HTTP request



Server

Preflighted

Browser



OPTIONS



Server



CORS approval



Actual request



Server

Preflight is a **browser CORS mechanism**, not a separate HTTP protocol.

122. Most Important Things to Remember

HTTP

↓

Application-layer request/response protocol

HTTPS

↓

HTTP + TLS

GET

↓

Read

POST

↓

Create/submit

PUT

↓

Replace

PATCH

↓

Partial update

DELETE

↓

Delete

401

↓

Authentication problem

403

↓

Permission problem

404

↓

Resource not found

429

↓

Rate limited

500

↓

Server error

CORS

↓

Browser cross-origin access control

OPTIONS

↓

Can be used for CORS preflight

Cookie



Browser-managed state

Authorization



Credentials/access token

TLS



Encryption + integrity + server authentication

HSTS



Tell browser to use HTTPS

HTTP/2



Multiplexed binary framing over TCP

HTTP/3



HTTP over QUIC

123. Backend Developer Learning Order

For Node.js backend development, study HTTP in this order:

1. HTTP fundamentals

↓

2. Request & Response

↓

3. Methods

↓

4. Headers

↓

5. Status codes

↓

6. URL / Params / Query

↓

7. Request body / JSON

↓

8. Cookies

↓

9. Authentication

↓

10. CORS

↓

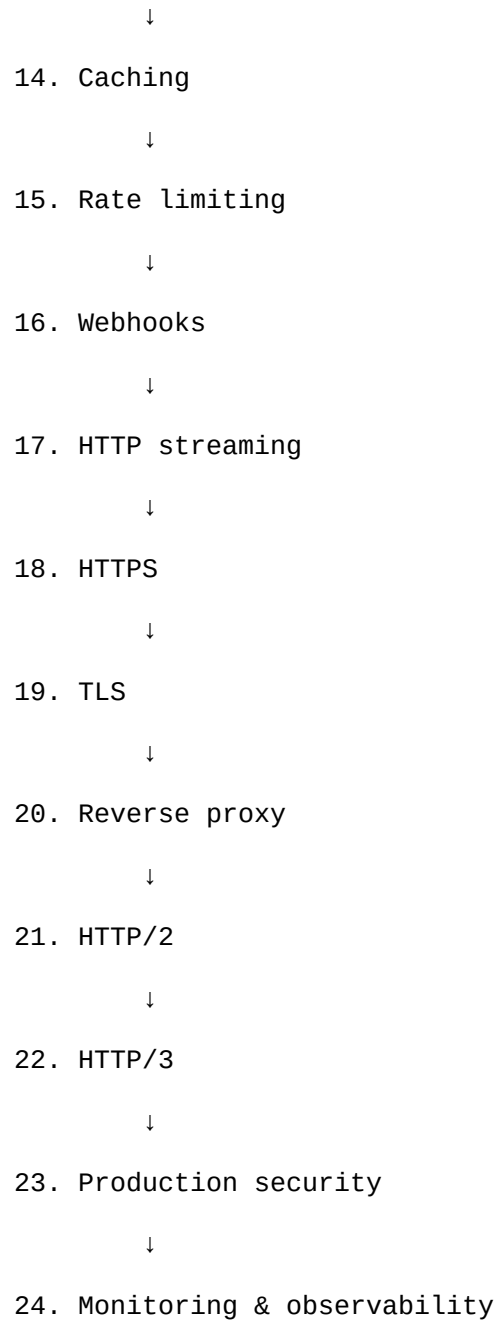
11. REST APIs

↓

12. Error handling

↓

13. Validation

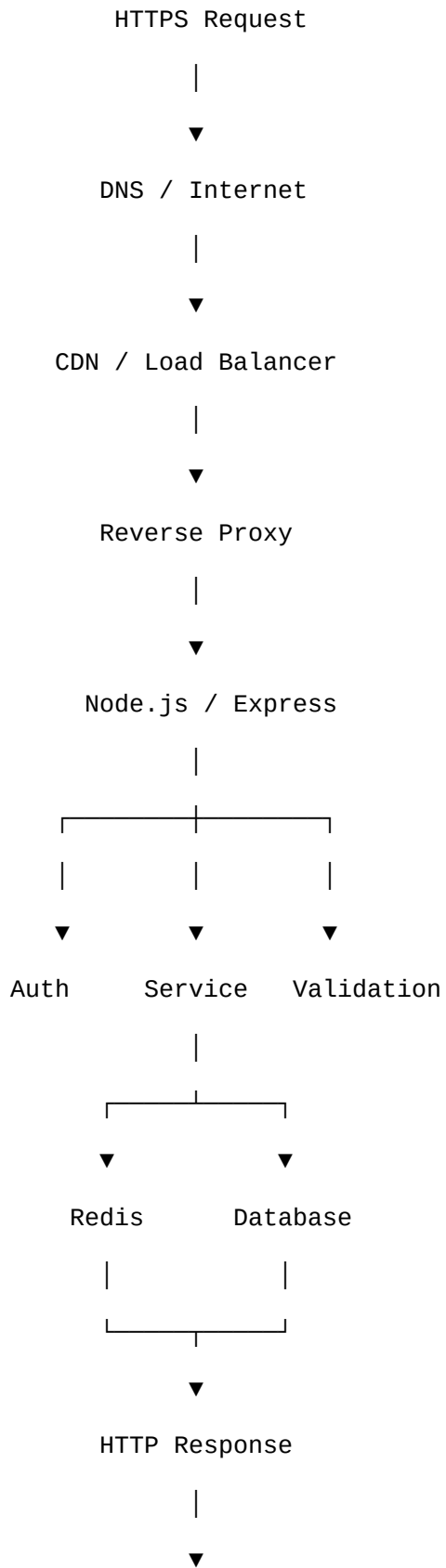


Final Backend Mental Model

CLIENT

|

|



CLIENT

Core idea: HTTP defines how your backend communicates. HTTPS secures that communication with TLS. As a backend developer, you should understand the complete journey from **DNS** → **connection** → **TLS** → **HTTP request** → **Express middleware** → **controller/service** → **database** → **HTTP response**.